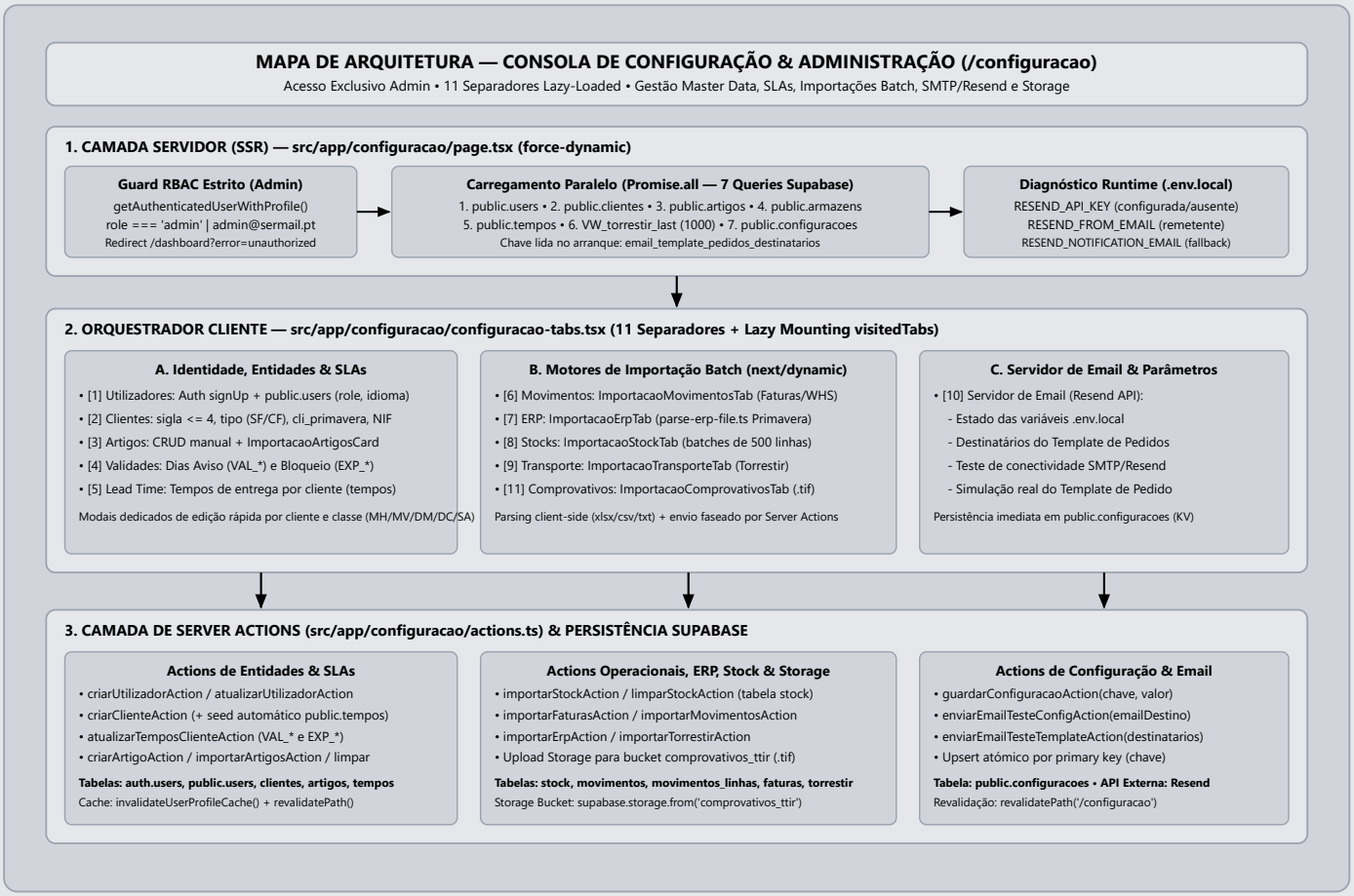
Documentação Funcional e Técnica — Página Configuração (`/configuracao`)

Módulo: Consola Central de Administração, Dados Mestres, SLAs Farmacêuticos, Motores de Importação Batch e Servidor de Email.

Rota: /configuracao | SSR: force-dynamic | RBAC: Exclusivo Admin | 11 Separadores (Lazy-Mounted) | Supabase Auth + Postgres + Storage + Resend

1. Diagrama de Arquitetura e Fluxo de Dados

O diagrama seguinte ilustra o fluxo completo desde a validação RBAC e carregamento paralelo no Server Component até aos 11 separadores cliente e respetivas Server Actions.



2. Carregamento no Servidor e Segurança RBAC (`src/app/configuracao/page.tsx`)

- **Controlo de Acesso Estrito:** Invoca `getAuthenticatedUserWithProfile()`. Se o utilizador não possuir `profile.role === 'admin'` (ou `user.email === 'admin@sermail.pt'`), é imediatamente redirecionado para `/dashboard?error=unauthorized`.
- **Execução Paralela (`Promise.all`):** Carrega num único ciclo assíncrono 7 fontes: `users`, `clientes`, `artigos`, `armazens`, `tempos`, `VW_torrestir_last` (limite de 1000 registos) e a chave `email_template_pedidos_destinatarios` em `public.configuracoes`.
- **Diagnóstico Seguro de Variáveis de Ambiente:** Inspeciona no servidor `RESEND_API_KEY`, `RESEND_FROM_EMAIL` e `RESEND_NOTIFICATION_EMAIL`, enviando ao cliente apenas flags booleanas e endereços remetentes em `emailEnvStatus`.

3. Matriz dos 11 Separadores (`src/app/configuracao/configuracao-tabs.tsx`)

Os separadores pesados recorrem a `next/dynamic` com `ssr: false` e montagem diferida controlada pelo estado `visitedTabs`:

#	Separador	Componente / Motor	Responsabilidade Funcional e Regras	Tabelas / Recursos
1	**Utilizadores**	`ConfiguracaoTabs` (nativo)	Registo em `auth.users` (`signUp`) + perfil em `public.users` (perfis `admin`, `gestor`, `user1`, `user2`, associação a `cliente_id`, idioma `PT/ES/EN` e estado `ativo`). Invalida a cache LRU de autenticação.	`auth.users`, `public.users`
2	**Clientes**	`ConfiguracaoTabs` (nativo)	Criação e edição de clientes depositantes: `sigla` (<= 4 caracteres), `tipo_cliente` (`SF` Sem Faturação / `CF` Com Faturação), código Primavera, NIF e morada. Inicializa automaticamente os SLAs em `public.tempos`.	`public.clientes`, `public.tempos`
3	**Artigos**	`ImportacaoArtigosCard` + CRUD manual	Registo manual de artigos, limpeza global de catálogo e importação em lote (`.xlsx`, `.csv`, `.txt`) com mapeamento de classe farmacêutica e conservação.	`public.artigos`
4	**Validades**	Modal de SLAs FEFO	Parametrização por cliente e por tipologia (**MH, MV, DM, DC, SA**) dos dias de **Aviso (`VAL_*`)** e dias de **Bloqueio (`EXP_*`)** que governam os alertas em `/stocks` e `/pedidos/criar`.	`public.tempos`
5	**Lead Time**	Painel `tempo_entrega`	Visualização e configuração dos tempos acordados de preparação e entrega logística por cliente depositante.	`public.tempos`
6	**Movimentos**	`ImportacaoMovimentosTab`	Importação assistida de movimentos de armazém e faturas em lote, alimentando cabeçalhos, linhas e faturação.	`public.movimentos`, `movimentos_linhas`, `faturas`
7	**ERP**	`ImportacaoErpTab`	Parsing especializado (`parse-erp-file.ts`) e integração de ficheiros exportados do ERP Primavera.	`public.movimentos`, `public.faturas`
8	**Stocks**	`ImportacaoStockTab`	Sincronização de fotografia de stock (`parse-stock-file.ts`) em lotes faseados de 500 linhas e limpeza de stock.	`public.stock`
9	**Transporte**	`ImportacaoTransporteTab`	Importação de ficheiros de tracking Torrestir (guias, portes, estados de entrega, volumes e incidências).	`public.torrestir`, `VW_torrestir_last`
10	**Servidor de Email**	Painel Resend & Templates	Diagnóstico de variáveis `.env.local`, configuração dinâmica de `email_template_pedidos_destinatarios` e envio de emails de teste (simples e template de pedido).	`public.configuracoes`, API **Resend**

#	Separador	Componente / Motor	Responsabilidade Funcional e Regras	Tabelas / Recursos
11	**Comprovativos**	`ImportacaoComprovativosTab`	Upload massivo e sincronização de comprovativos digitalizados (`.tif` / `.tiff`) para o bucket de Storage.	Bucket Storage `comprovativos_ttir`

4. Server Actions Principais e Regras de Persistência (`src/app/configuracao/actions.ts`)

- `criarUtilizadorAction` / `atualizarUtilizadorAction` : Criam/atualizam o registo em `auth.users` e `public.users` e invocam `invalidateUserProfileCache(userId)` para que alterações de perfil ou cliente tenham efeito imediato.
- `criarClienteAction` : Além de inserir o cliente em `public.clientes` , executa um *seed* automático na tabela `public.tempos` com `VAL_* = 180` dias e `EXP_* = 60` dias para todas as classes (MH, MV, DM, DC, SA).
- `atualizarTemposClienteAction` : Atualiza os 10 limiares de validade em `public.tempos` e revalida `/configuracao` .
- `guardarConfiguracaoAction(chave, valor)` : Valida permissões de administrador e executa `upsert({ chave, valor, updated_at }, { onConflict: 'chave' })` na tabela Key-Value `public.configuracoes` .
- `enviarEmailTesteConfigAction` & `enviarEmailTesteTemplateAction` : Testam em tempo real o transporte HTTP para `https://api.resend.com/emails` com o remetente configurado e a lista de destinatários do template de pedidos.